

SHL Assessment Recommender — Approach Document

AI Intern Take-Home Assignment — Aman Raj

1 Architecture Overview

The service is a stateless FastAPI app with three layers: (1) a **retrieval layer** over the SHL Individual Test Solutions catalog, (2) an **LLM decision layer** (Groq, LLaMA 3.3 70B) that picks one of five actions per turn — **clarify**, **recommend**, **refine**, **compare**, **refuse** — constrained to JSON output, and (3) a **validation layer** in plain Python that resolves every recommended name back against the real catalog before it reaches the response, so the LLM can never cause a hallucinated name or URL to be returned. Each `/chat` call rebuilds everything from the full message history it receives; no server-side state is kept, matching the stateless contract in the spec.

2 Data Preparation

The provided catalog JSON (377 items) included 7 pre-packaged “Job Solution” bundles (e.g. *Entry Level Sales Solution*), identifiable by a `-solution/` URL slug. These were excluded, leaving 370 Individual Test Solutions. Each item’s category tags (e.g. “Personality & Behavior”) were mapped to SHL’s single-letter `test_type` codes (A/B/C/D/E/K/P/S) to satisfy the response schema.

3 Retrieval: BM25, not embeddings

Candidate assessments are retrieved with BM25 keyword search (`rank_bm25`) over a composed text (name + description + category + job levels) per item, rather than dense embeddings. Two reasons drove this: the catalog is small (370 items), so retrieval quality is dominated by lexical overlap rather than semantic nuance, and queries frequently contain exact technical tokens (`.NET`, `OPQ32r`, `AWS`) that BM25 matches precisely. Just as importantly, it avoids downloading embedding-model weights at container start, which matters for cold-start reliability on free hosting within the assignment’s 2-minute wake-up allowance.

One deliberate augmentation: the base personality instrument *Occupational Personality Questionnaire OPQ32r* is always force-included in the candidate pool, and the prompt instructs the model to prefer it over narrower “Report”/“Profile” variants (which are interpretive outputs built on OPQ32r data, not substitutes). This was added after observing that BM25 alone tends to rank name-matching report variants (e.g. *OPQ Leadership Report*) above the base instrument for role-specific queries, while SHL’s own sample answers consistently include the base instrument.

4 Agent Design and Prompt

The system prompt is deliberately short (~470 tokens) and rule-based rather than few-shot, to keep per-call token cost low (see §5). It encodes: (a) the five-action decision, (b) a hard constraint that recommended names must come only from the supplied candidate list, (c) an instruction to substitute the nearest relevant items rather than refuse when no test exactly matches a named skill (e.g. no dedicated “Rust” test), and (d) a turn-budget rule — at most two clarifying questions before committing to a shortlist.

Two safety nets are enforced in code, not trusted to the model: (1) if the model still chooses **clarify** on the third user turn, the code forcibly converts the action to **recommend** using the top retrieved candidates, guaranteeing convergence within the evaluator’s 8-turn cap regardless of model judgment; (2) every recommended name is resolved against the live catalog via exact/substring match, and unresolved names are silently dropped and backfilled from the top BM25 candidates, so the “recommendations” array is never inconsistent with a committed **recommend/refine** reply.

5 Reliability Engineering (Rate Limits and Timeouts)

Groq’s free tier is rate-limited on tokens-per-minute, not just request count. An early version sent an ~1100-token system prompt plus 18–25 candidate descriptions per call and hit 429s consistently. The fix was to cut the system prompt to ~470 tokens, shorten candidate descriptions, and cap the candidate pool, bringing typical per-call cost down to ~1500–1800 tokens. Retries on 429 read Groq’s rate-limit response headers (`x-ratelimit-reset-tokens`) for an accurate backoff instead of guessing, and the whole retry loop is bounded by an 18-second wall-clock deadline — well inside the assignment’s 30-second per-call timeout, including FastAPI/network overhead. If the LLM is still unreachable after the deadline, a retrieval-only heuristic fallback returns a schema-valid response (a generic clarifying question on turn 1, or the top BM25 candidates as a shortlist afterwards) instead of a 500 — protecting the “schema compliance on every response” hard eval even under a full provider outage.

6 Evaluation

A local harness (`tests/run_traces.py`) replays the 10 provided sample traces turn-by-turn against a running server and reports a rough Recall@10 against each trace’s expected shortlist. This is a development proxy only — it replays the literal scripted user turns, whereas the real evaluator uses an LLM-simulated user that can answer out of order or self-correct. Across iterations, mean recall on the 10 public traces improved from 0.24 (early version, aggressive candidate trimming, no OPQ32r preference) to ~0.38–0.43 (final version, verified both locally and against the deployed Render endpoint). Remaining gaps are concentrated in traces expecting many near-duplicate catalog entries (e.g. both “MS Word (New)” and “Microsoft Word 365 (New)”), which the agent under-recommends by design (it avoids listing redundant variants), and traces with no close catalog match for a specific named technology, where the agent substitutes reasonably but doesn’t recover the exact expected name.

7 What Didn’t Work

- **Dense embeddings via sentence-transformers/FAISS** were tried first but abandoned: the sandbox development environment couldn’t reach Hugging Face to download model weights, and given the small catalog size, BM25 was judged unlikely to underperform embeddings enough to justify the added cold-start risk on free hosting.
- **Gemini (gemini-2.5-flash)** was the original LLM choice, but its free tier caps at 5 requests/minute without billing enabled, which is incompatible with an 8-turn evaluator running many conversations back to back. Switched to Groq (LLaMA 3.3 70B), whose free tier resets per-minute with no daily billing requirement.
- **An early retry policy** used a fixed retry count with up to 12s backoff per attempt, which could exceed 30s in aggregate and caused client-side read timeouts during testing. Replaced with the wall-clock-deadline approach described in §5.

8 Use of AI Tools

This project was built interactively with Claude (Anthropic) as an AI pair-programmer: Claude generated the initial architecture and code across iterations, ran local smoke tests, and proposed fixes for issues surfaced during testing (rate limiting, timeout budgeting, retrieval quality). All design decisions, trade-offs, and debugging (e.g. diagnosing the TPM rate-limit root cause, choosing BM25 over embeddings, the OPQ32r preference rule) were reviewed and directed by the author, who can walk through and defend every part of the codebase.